CS 2160: Performance Demo

Instructor: Brandon Collins

https://github.com/brandon-is-coding/c_benchmark

1 Optimizing a Program

Fundamentally, there are two ways to optimize a program. The first is *algorithm design*, that is given a problem, what is the best approach that will execute the fastest and using the fewest resources? Once an algorithm is selected, one has to provide an implementation that runs on a physical circuit we call a processor. Every level of abstraction in this process could be the topic of its own class (and we do offer classes at UCCS for most of them!): algorithm design, programming in a high level language, compiling a program to machine code, allocating the program physical resources (This is the operating system's job!), and finally executing a program as digital logic on a modern processor. For this demo, we will suppose that the algorithm has already been selected, and we will focus on how to check to see how well the program will run given our knowledge of computer architecture and assembly language.

2 Ten Million Additions

Today we will investigate a seemingly simple question: How long does it take a computer to perform 10 million additions? Here is a more detailed specification of this benchmarking problem:

- Two arrays (A,B) of 32 bit integers will be allocated by the bench-marker
- We must write a function `vec_add` that computes an array C such that: C[i] = A[i] + B[i].
- All memory allocation and deallocation related to C will be timed as part of the benchmark
- A,B will be of length 10,000 and the function `vec_add` will be called 1000 times, resulting in 10 million total additions.

3 Tip #1: How fast *should* a program run?

Before we sit down and try to figure out if a particular implementation is good or not, we should have an idea of how fast we think a program should run. For example, if we reason that a computer can do X number of operations in Y seconds, and the real performance is dramatically slower then Y, then it is likely something is going wrong at the low levels. For example, let us consider how fast a computer might be able to do 10 million additions. This demo is being run from an m4 macbook air with an (optimistic) clock rate of about 4.5 gHz. Throughout the semester, we have assumed that

Implementation	C exe time	Julia exe time	Memory
`vec_add1`	0.016735	0.090319	62.531 MiB (96% GC)
`vec_add2`	0.013548	0.000510	64.031 KiB
`vec_add3`	0.010484	0.004248	64.062 KiB

Table 1: Results for Benchmark1 (ran on m4 Mac). Time is in seconds and memory usage is reported by the Julia implementations.

we will do one instruction per clock cycle, so if that holds, how long should the additions take? To do this, we simply divide the number of operations we want to do by the number of operations per second:

$$\frac{10 * 10^6}{4.5 * 10^9} \approx 0.00222 \text{ s} = 2.22 \text{ ms} \tag{1}$$

So, if we write an implementation and it performs dramatically worse then 2ms, either something is wrong with the implementation, one of our assumptions is flawed, or there are other constraints on the program. We will see examples of all three of these cases, but for now the important part is that we have an approximate point of comparison with a real performance measurement.

So, we just need to provide an a c implementation that matches the problem spec. Here is a first attempt:

```c
int* vec_add1(int* A,int* B,int n){
   int *C = malloc(sizeof(int) * n);
   for(int i=0;i<n;i++){
      C[i] = A[i] + B[i];
   }
   return C;
}
```

This implementation is fairly straightforward, we simply use a for loop to go over all indexes and simply add C[i] = A[i] + B[i]. Additionally, as the problem requires, we allocate the memory for C, we use a malloc to allocate array C. The results for this implementation are the first row Table 1. We see that it runs in 16ms, 8 times slower than predicted. Something is certainly wrong, but what?

4 Tip #2: Allocations in the Hot Path

Without even inspecting the assembly, we can see that we have introduced a malloc into the hot path of our code. We say that a *hot path* of a program is the portion that is frequently executed. The hottest part of our program is inside the for loop, which will run 10 million times, but outside the forloop the malloc will run 1000 times. Therefore, our implementation does not only make 10 million additions, but also 1000 allocations. In general, **memory allocations are slow**. Therefore, one of the lowest hanging fruit is to simply move the allocations out of the hot path. To do this, we write

vec_add2:

```
int* vec_add2(int* A,int* B,int* C,int n){
   for(int i=0;i<n;i++){
      C[i] = A[i] + B[i];
   }
   return C;
}
```

Here we allocate C outside of the function and simply rewrite its contents 1000 times. For `vec_add2`, we allocate and deallocate C a single time. The performance result seen in Table 1 is a $\approx 20\%$ performance speed up.

This is a fairly simple example, and we are writing the code in c. c is one of the very few programming languages with *manually managed memory*. This means that we must explicitly allocate when memory is allocated or freed. The majority of popular languages today (Java, python, JavaScript, Go, C#...) have *automatically managed memory*. This means that they have a *Garbage Collector* that periodically wakes up, looks for memory with no active reference or pointer, and then cleans it up. From a programmer's perspective, this leads to languages that are (arguably) easier to write in, but it also means allocations are no longer explicit. Consider this re-implementation of `vec_add1` in a garbage collected language called Julia:

```
1 function add_vec1(A,B,n)
2    C = Vector{Int32}(undef, n)
3    for i in 1:n
4       C[i] = A[i] + B[i]
5    end
6    return C
7 end
```

Here the allocation happens on line 2, however it is up to the programmer to know that this statement is a memory allocation under the hood. More importantly, it is up to the programmer to understand the consequences of placing an allocation in any given place. The performance of Julia is also shown in Table 1, and Julia provides nice tools to benchmark the memory usage of the program. The fourth column of the is the reported results from the memory benchmarking, and the difference between `vec_add1` and `vec_add2` is staggering. The garbage collector took 96% of execution time!

5 Tip #3: Multiple Issue

Now that we have implemented `vec_add2`, we can be relatively sure that the additions are now the main focus of our program. But something bizarre is happening as Julia appears to be an order of magnitude faster than c. This is strange, as c and other fully compiled languages are often considered

to be the fastest. Further, c being the "fastest" language does have some merit:

- Fully compiled languages like c enforce that types are known at compile time, and therefore c can produce efficient machine code at compile time.
  - This in contrast to an interpreted language like python, where the types may not be known until run time. Essentially, for a function like f(x,y) = x + y, python must figure out if it is doing integer addition or floating point addition at runtime, which is inherently slower.
- Manual memory mangament means the programmer has absolute control over when allocations and de-allocations happen

How is c failing behind in this benchmark when compared with Julia? First, we can ask do we expect Julia to be fast? It is a dynamically typed language with a garbage collector and just-in-time compiler. Much like a fully compiled language, a just-in-time compiler is also capable of producing high quality machine code, the only difference is that julia does compilations the first time a function is called. Second, the implementation of `vec_add2` is allocation free, so one would not expect having a garbage collector or not to significantly impact the functions performance. This is supported by the fact that Julia's benchmarking did not report any garbage collector activity. Regardless, we can treat Julia's performance as a benchmark to meet or beat, as there is no technical reason for c to be slower.

To examine why c is slow, we can consider the compiler. The above experiments were done using the command:

```
gcc benchmark.c
```

This seems straightforward enough, we simply call the standard c compiler on the c program. One subtlety of the gcc compiler is there there are various *optimization levels*. The default optimization level is -O0, where gcc performs remarkably few optimizations. So, at -O0 we expect the compiler to relatively faithfully implement our C program. Previously, we assumed that a program would run one instruction per clock cycle. We know this to be untrue, in practice modern processors usually have multiple compute units for various tasks. Further, we know that branches incur significant overhead as the branch behavior must be predicted and speculated. It often happens that the compiler will *unroll* a loop so that the functional units of a processor can saturate with computations as a branch calculation is being done. At -O0 it is true that the compiler is not unrolling the loop (we can check `benchmark_O0.s`), so what if we unroll the loop manually. This leads to `vec_add3`:

```
int* vec_add3(int* A,int* B,int* C,int n){
   for(int i=0;i<n;i+=2){
      C[i] = A[i] + B[i];
      C[i+1] = A[i+1] + B[i+1];
   }
   return C;
```

```
}
```

Here, I have unrolled the loop two times. A few interesting observations:

1. Unrolling the loop resulted in roughly a 30% speedup in C
2. Unrolling the loop caused a performance loss in Julia!
3. Here I unrolled it twice - unrolling it other amounts had about the same effect (on Mac!)
4. (Unofficial) Spec sheets state that the M4 processor likely has two ALUs supporting addition, it is possible that this is related to why unrolling it twice helps on this processor. We can not say for sure that this is the reason though!

6 Tip #4: The Hunt for SIMD

Even with `vec_add3`, c is still much slower than julia, so where are we losing performance? To get the answer let's compare the assembly outputs of Julia's `vec_add2` and c's `vec_add3`. Starting with c, we (roughly) find the main body of `vec_add3`:

```
ldr  x8, [sp, #24]
ldrsw x9, [sp]
ldr  w8, [x8, x9, lsl #2]
ldr  x9, [sp, #16]
ldrsw x10, [sp]
ldr  w9, [x9, x10, lsl #2]
add  w8, w8, w9
ldr  x9, [sp, #8]
ldrsw x10, [sp]
str  w8, [x9, x10, lsl #2]
ldr  x8, [sp, #24]
ldr  w9, [sp]
add  w9, w9, #1
ldr  w8, [x8, w9, sxtw #2]
ldr  x9, [sp, #16]
ldr  w10, [sp]
add  w10, w10, #1
ldr  w9, [x9, w10, sxtw #2]
add  w8, w8, w9
ldr  x9, [sp, #8]
ldr  w10, [sp]
add  w10, w10, #1
str  w8, [x9, w10, sxtw #2]
b  LBB2_3
```

- This is ARM assembly, a few tips:
  - "st" means store, similar to how we used sw
  - "ld" is load, similar to lw
  - "b" is branch, here b is used how j would be in RISCV
- Overall it appears doing a bunch of loads, a few adds, then a store as we would expect. However, From a RISCV perspective we would expect 2 loads per add and then one store, but this procedure seems to have more instructions then this. This is a tell that perhaps this assembly is not the most optimized.

Now the Julia's assembly:

```
ldp  q0, q1, [x5, #-32]
ldp  q2, q3, [x5], #64
ldp  q4, q5, [x6, #-32]
ldp  q6, q7, [x6], #64
add.4s  v0, v4, v0
add.4s  v1, v5, v1
add.4s  v2, v6, v2
add.4s  v3, v7, v3
stp  q0, q1, [x7, #-32]
stp  q2, q3, [x7], #64
subs x4, x4, #16
```

- Group of loads, then adds, then stores
- "ldp" is load pair, loads two registers at once
- "stp" is store pair
- most critically add.4s is an instruction not seen in the c's assembly (uses normal add).

So, what is add.4s doing? All instruction we have seen so far are **Single Instruction Single Data (SISD)**. This instruction do one operation between the operands. One way modern processors accelerate computations is **Single Instruction Multiple Data (SIMD)**. Here, we use one instruction to indicate that we want multiple operations done between multiple pieces of data. The way this works is that we use *vectorized registers*. In the case of the M4 processor, it support's ARM's NEON SIMD extension, which supports 128 bit registers. These are capable of four simultaneous (32 bit) integer additions at one time. So, add.4s is doing 4 adds *at once*, treating the v registers as having 4 consecutive integers in them. Additionally, we can see that the program is using vectorized loads and saves. The overall loop here does twice as much work as the unrolled version from C, and does so

with many fewer instructions. Supposing the processor can support the bandwidth and computation for these loads, computations, and saves, then one would expect this program to run much faster the the c version. This leads to two questions:

1. Supposing we do 4 adds per cycle, how fast should the program run?
    - Following the same math as before: $\frac{10*10^6}{4*4.5*10^9} = 0.000555$ or about 555 microseconds!
    - This lines up very closely with julia's observed performance!
2. How do we ensure the c implementation uses the SIMD instructions?
    - option 1: utilizing compiler optimization flags
    - option 2: manually programming with SIMD intrinsics

To address each point, we see that julia benchmarks very close to the timing if it performed exactly 4 additions per cycle. Many factors could contribute to this holding true or not, but with the given information it seems likely that this is at least close to the true behavior of the processor.

Moving back to the c side of things, we have two options to ensure that we get SIMD instructions. The first and easiest is to simply switch the flag from -O0 to -O3, the most aggressive optimization setting. We can inspect `benchmark_O3.s` and find the exact same SIMD instructions as appeared in julia's output. So, with the right flags the c compiler will agree with the julia compiler, and likely both programs will benchmark at roughly the same speed. Unfortunately, aggressive optimization settings also optimize the benchmark code (inspecting `benchmark_O3.s` reveals that `vec_add` calls have been removed). This means that the benchmark code cannot report a time for c, but this is an issue with the benchmark, then the compiler. In non-trivial settings, the compiler will not remove your important code.

Option 2 for ensuring SIMD output is using intrinsics. Intrinsics let us explicitly vectorize code in high level languages. Essentially, the main idea of intrinsics is that we do not make the compiler infer that the for loop can be vectorized, rather we use a 128 bit data type and explicitly perform vector additions in the for loop. This explicit form of vectorization is sometimes necessary when the compiler cannot infer that it is possible to vectorize code. In summary:

- If you think the code *should* be vectorized, check the assembly for the presence of SIMD instructions.
- Use compiler flags to ensure optimized output
- If the compiler cannot vectorize code you think should be vectorized, use intrinsics to manually do the vectorization!